Wireshark Lab

Matthew Blanchard

CYEN 3331-01

Dr. Lyford

2/25/25

Introduction

The objective of these laboratory exercises is to establish a robust framework for network troubleshooting and forensic analysis within the Wireshark environment. Initial steps involve the creation of a "Troubleshooting" configuration profile, which serves as a centralized repository for specialized filters, coloring rules, and custom data columns. This tailored environment allows for the rapid identification of protocol inefficiencies and anomalies that standard default settings may obscure. Through sequential labs, the focus shifts from basic setup to the sophisticated extraction of timing metadata and hierarchical protocol statistics.

Executive Summary

The technical analysis successfully identifies several critical network behaviors across multiple trace files, ranging from high-latency DNS responses to suspicious botnet-like traffic. Performance bottlenecks were localized by implementing Delta Time and TCP Delta columns, which provide granular visibility into round-trip path latency and individual conversation delays. Security posture was enhanced through the deployment of automated coloring rules designed to flag DNS error codes and responses containing an unusually high number of resource records. Furthermore, the use of I/O graphs and endpoint statistics allowed for the precise identification of "top talkers" and sudden throughput drops during file downloads.

Lab 1

Objective:

Create and manage a dedicated "Troubleshooting" configuration profile to store custom filters, columns, and coloring rules for consistent use throughout the course.

Instructions:

1. Open Wireshark.
2. Right-click on the Profile column in the Status Bar and select New.
3. Rename the new profile to Troubleshooting and click OK.
4. Verify the new profile name appears in the Status Bar.
5. Right-click the Profile column and select Manage Profiles to confirm the Troubleshooting profile is selected.
6. Double-click the hyperlink for the profile directory to view personal configuration files.

Observations:

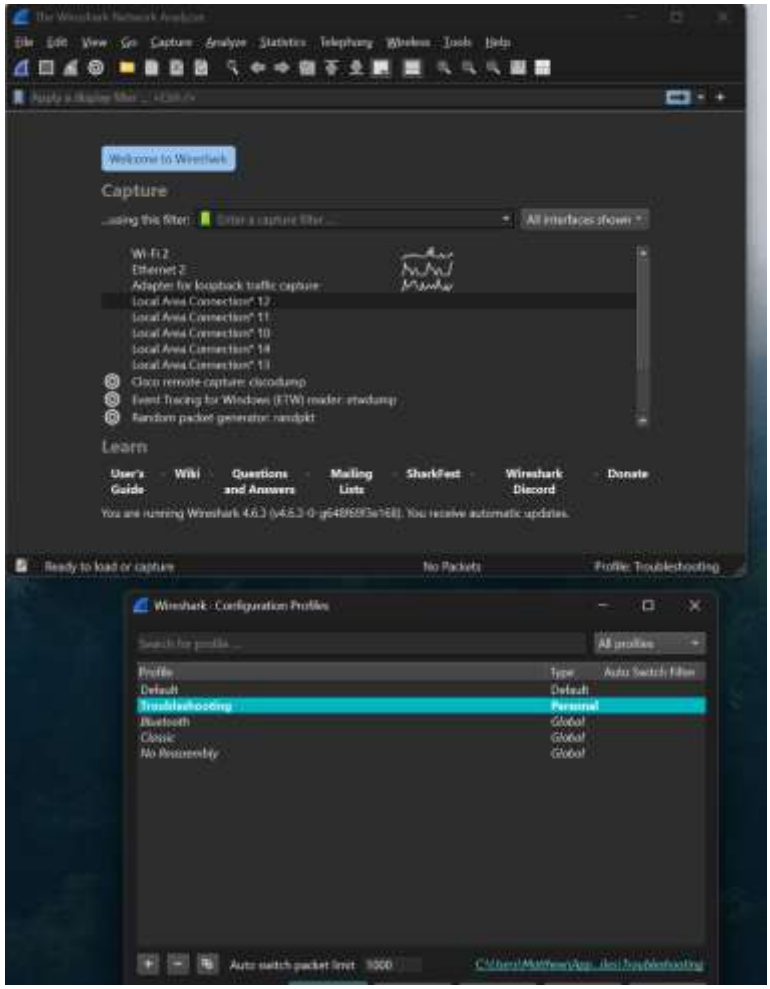

Figure 1

Profile successfully created. (Figure 1).

Lab 2

Objective:

Identify network latency by adding custom delta-time columns and configuring coloring rules to highlight significant delays in TCP streams.

Instructions:

1. Open lab-timeanalysis.pcapng using the Troubleshooting profile.
2. Verify the Time Display Format is set to Seconds Since Beginning of Capture via the View menu.
3. Right-click the Frame section of Packet #1 and select Expand Subtrees.
4. Locate the field [Time delta from previous displayed frame], right-click it, and select Apply as Column.

5. Drag the new column heading to the left, placing it next to the default Time column.
6. Right-click the new column heading, select Edit Column, and rename it to Delta Time.
7. Enable TCP timestamps by right-clicking the TCP header in packet 29, navigating to Protocol Preferences, and ensuring Calculate conversation timestamps is enabled.
8. Expand the [Timestamps] section at the end of the TCP header in Packet 29.
9. Right-click the [Time since previous frame in this TCP stream] line and select Apply as Column, then rename it TCP Delta.
10. Create a coloring rule for TCP delays by right-clicking the TCP stream delta line in Packet 32 and selecting Colorize with Filter > New Coloring Rule.
11. Configure the rule with the name T-TCP Delays, filter tcp.time_delta > 1, black foreground, and orange background.
12. Refine the filter to tcp.time_delta > 1 && tcp.flags.fin==0 && tcp.flags.reset==0 to exclude connection terminations.
13. Sort the TCP Delta column from high to low to identify the largest delays.

Observations:

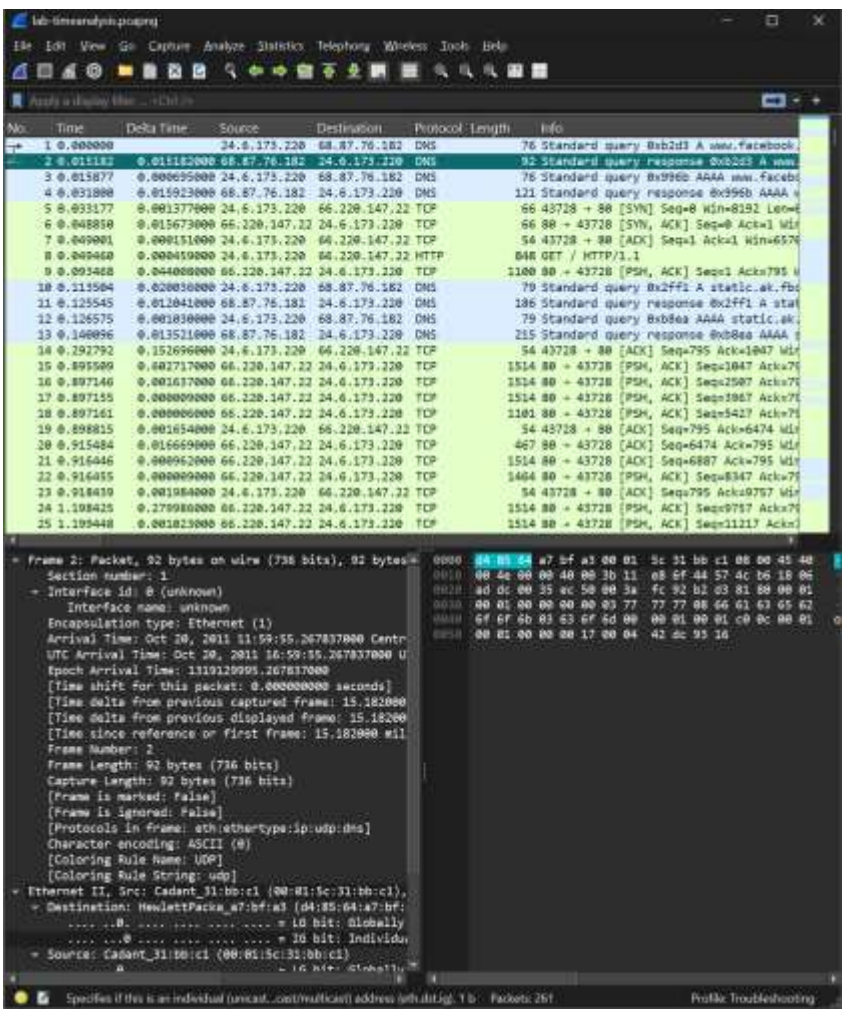

Figure 2.1

Time Delta for packets 2, 4, 11, and 13 are as follows (Figure 2.1):

- Packet 2 DNS Response Time: 15.18 ms
- Packet 4 DNS Response Time: 15.92 ms
- Packet 11 DNS Response Time: 12.04 ms
- Packet 13 DNS Response Time: 13.52 ms

Figure 2.2

The round trip path latency seen during the TCP connection between 24.6.173.220 and 66.220.147.22 (Frames 29 – 32) is 19.21 ms (Figure 2.2).

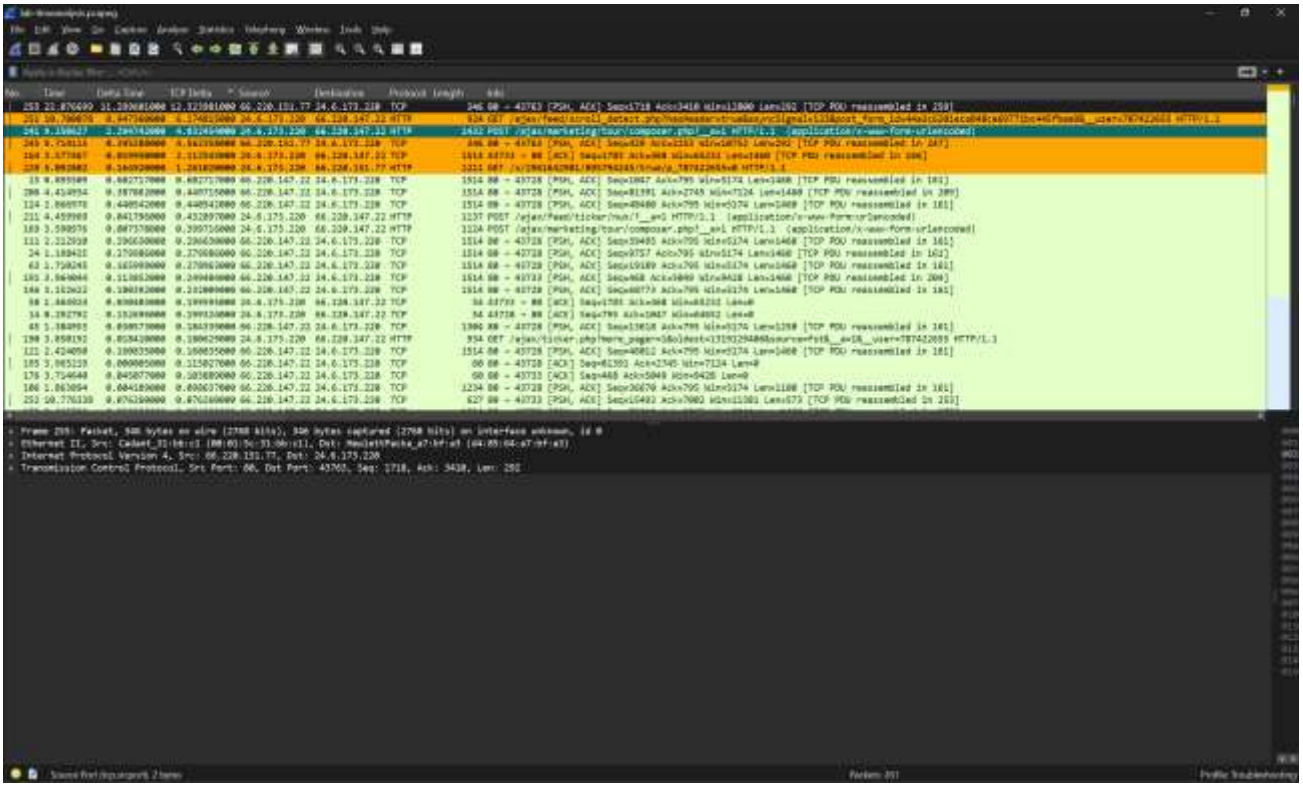

Figure 2.3

The largest delays in this trace file are as follows (Figure 2.3):

- Packet 255 with a TCP delta of 12.32 seconds
- Packet 251 with a TCP delta of 6.18 seconds
- Packet 241 with a TCP delta of 4.83 seconds
- Packet 245 with a TCP delta of 4.56 seconds
- Packet 164 with a TCP delta of 2.11 seconds
- Packet 229 with a TCP delta of 1.26 seconds

The high TCP Delta values in these top packets, some exceeding 10 seconds, likely indicate significant network congestion or a server-side bottleneck where the host is struggling to process requests. Additionally, these delays could be caused by packet loss leading to lengthy retransmission timeouts, especially given the "TCP PDU reassembled" notes which suggest fragmented or out-of-order data delivery.

Lab 3

Objective:

Locate the most active hosts and protocols on a network by analyzing IPv4 endpoints, TCP conversations, and the protocol hierarchy.

Instructions:

1. Open lab-gentraffic.pcapng using the Troubleshooting profile.
2. Select Statistics | Endpoints and navigate to the IPv4 tab.
3. Sort by Tx Bytes to identify the host sending the most data.
4. Select Statistics | Conversations, navigate to the TCP tab, and sort by Bytes to find the most active conversation.
5. Select Statistics | Protocol Hierarchy to identify the most active UDP and TCP-based applications.

Observations:

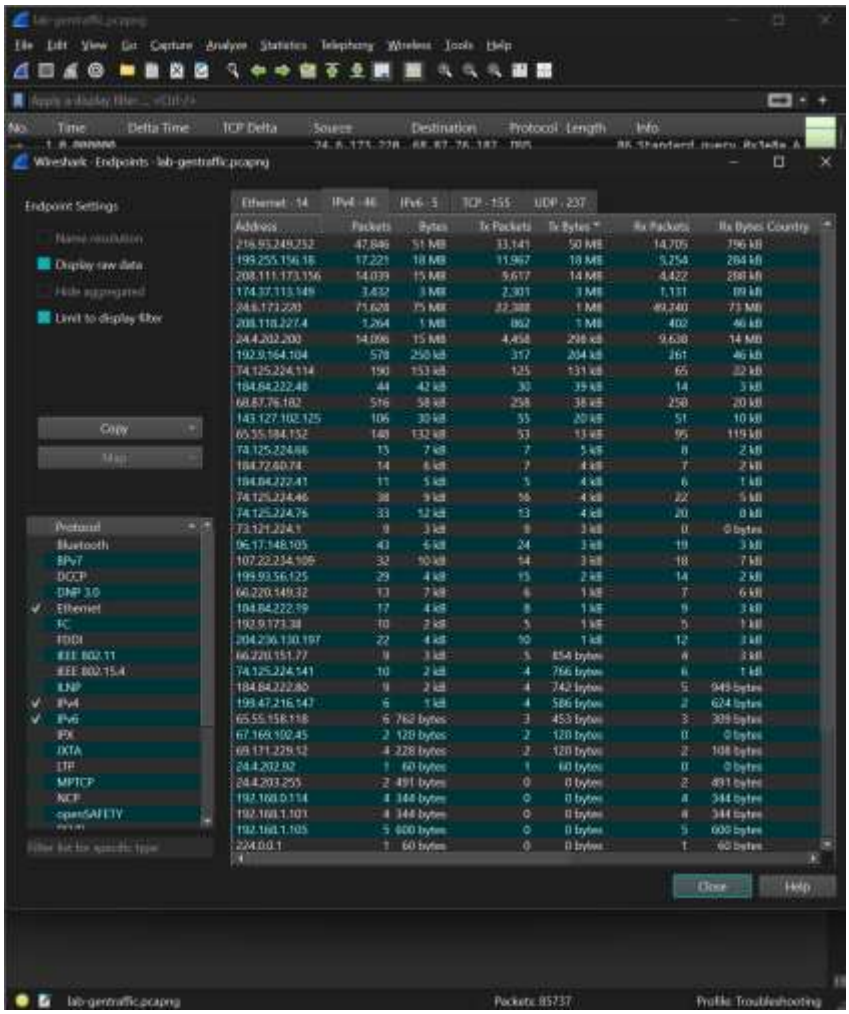

Figure 3.1

The most active talker IPv4 host is 216.93.249.242 (Figure 3.1).

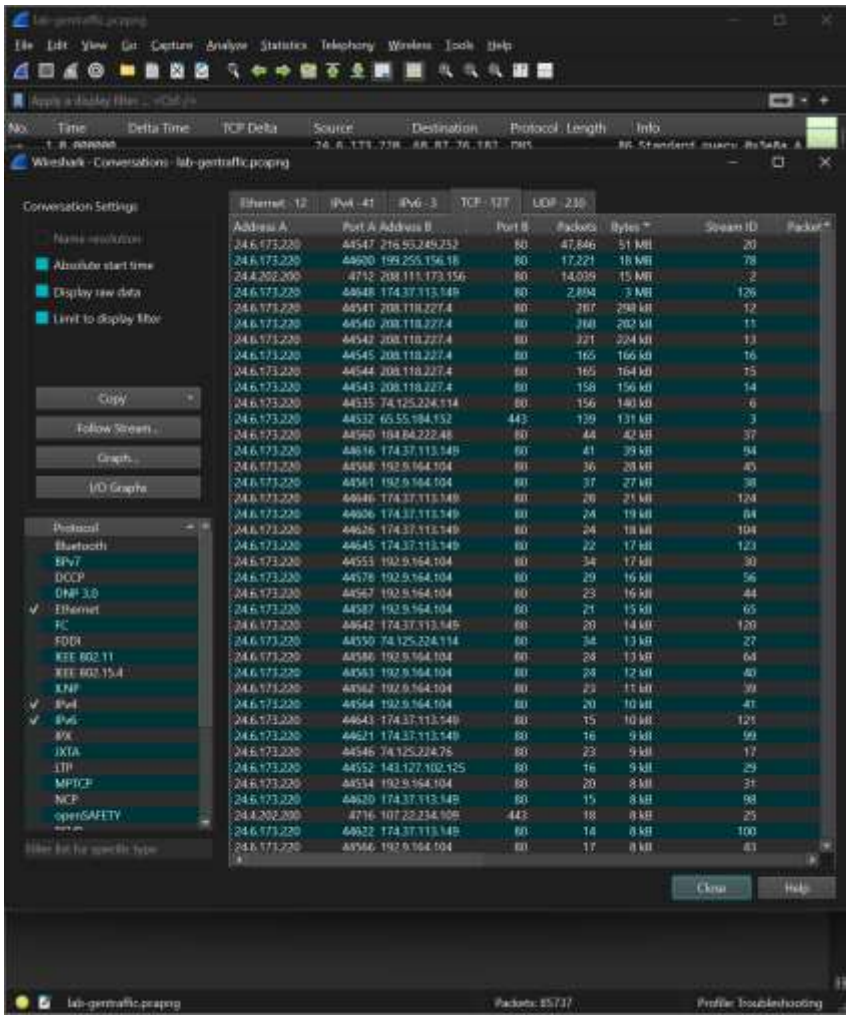

Figure 3.2

The most active TCP conversation by byte size is between 24.6.173.220:44547 and 216.93.249.252:80 (Figure 3.2).

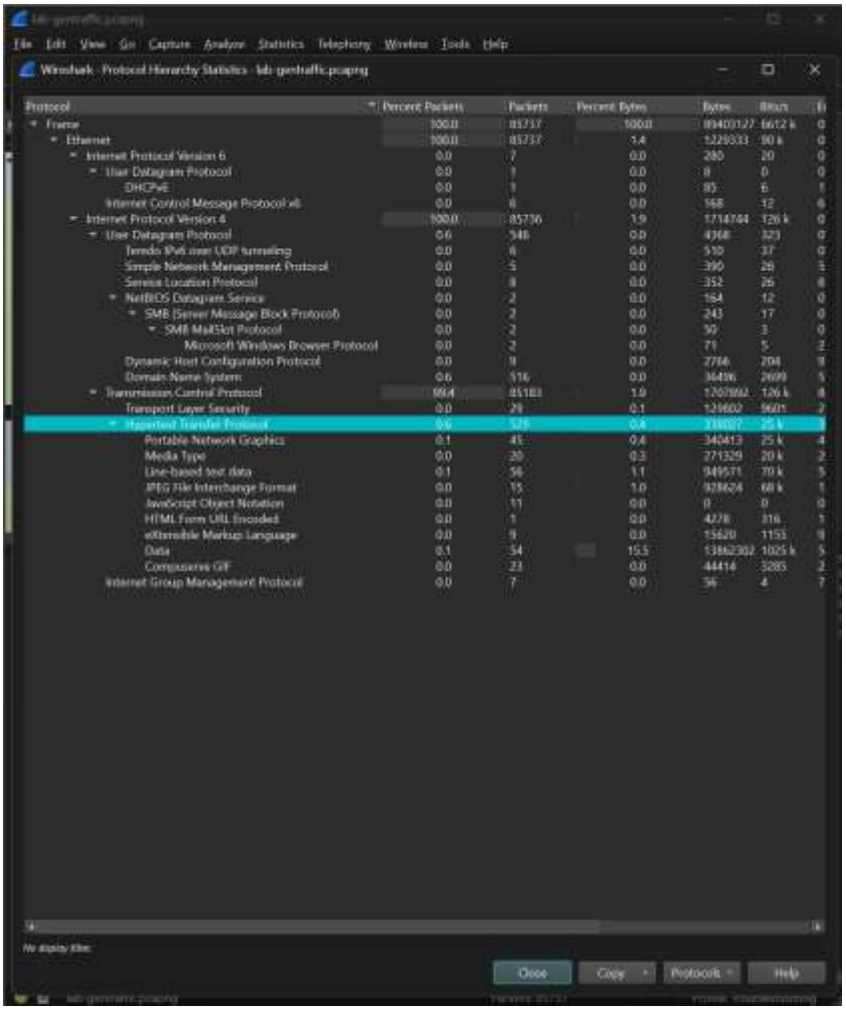

Figure 3.3

The most active UDP-based application is DNS with 516 packets. The most active TCP-based application is HTTP with 529 packets (Figure 3.3).

Lab 4

Objective:

Utilize I/O graphs to visually spot sudden performance drops and throughput issues within a packet capture.

Instructions:

1. Open lab-iograph.pcapng using the Troubleshooting profile.
2. Verify there is only one conversation by checking Statistics | Conversations.
3. Open Statistics | I/O Graph and observe the All packets line for sudden throughput drops.

4. Click on low points in the graph to jump to corresponding packets in the trace file for further investigation.

Observations:

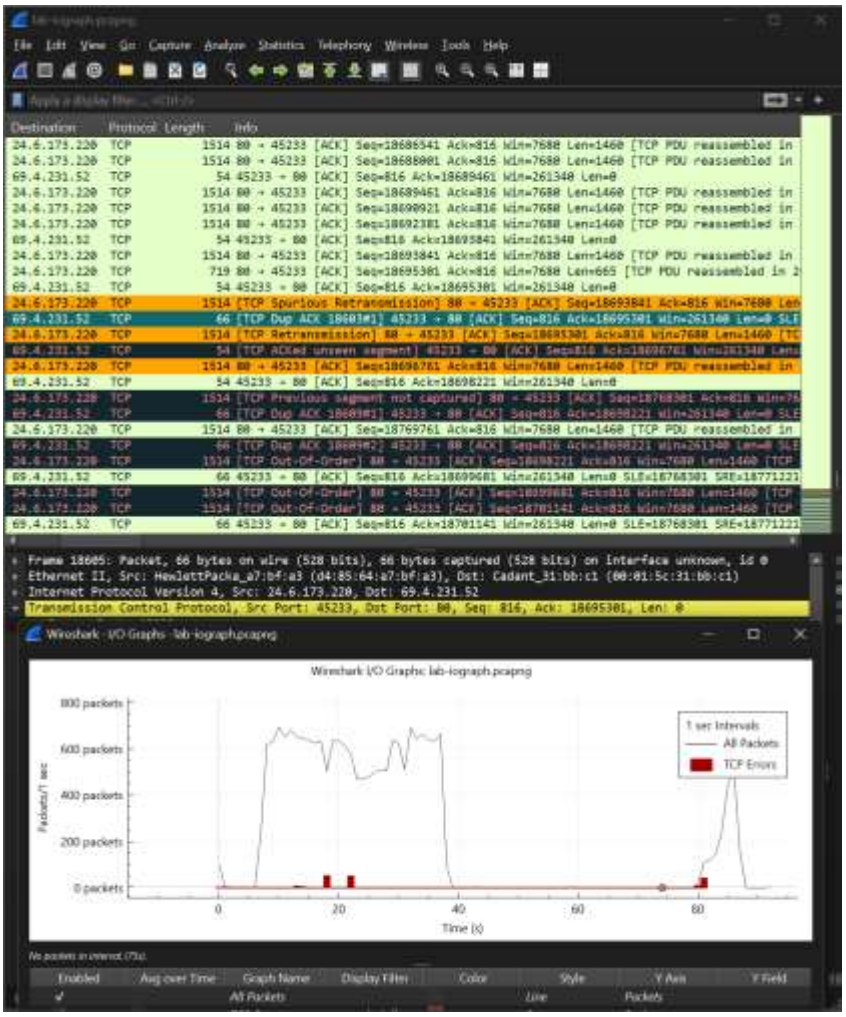

Figure 4.1

When the throughput drops in the trace file, Wireshark indicates conditions including out-of-order packets, retransmission errors, duplicate ACK, and incorrect segments (Figure 4.1).

Lab 5

Objective:

Build custom coloring rules based on specific DNS fields to quickly detect server failures and suspicious responses with unusually high answer counts.

Instructions:

1. Open lab-dns-errors-partial.pcapng.
2. Expand the Domain Name System (response) section in Packet 5.
3. Right-click the Reply Code field and select Colorize with Filter > New Coloring Rule.
4. Configure the rule: Name T-DNS Error Responses, Filter dns.flags.rcode > 0, black foreground, and orange background.
5. Create a second rule for suspicious DNS responses by right-clicking the Answer RRs line.
6. Configure the rule: Name S-DNS Suspicious (High DNS Response Count), Filter dns.count.answers > 10, black foreground, and dark orange background.
7. Ensure both rules are enabled and placed at the top of the coloring rules list.
8. Open lab-sec-sickclient.pcapng and observe if any packets are highlighted by the new "Suspicious" rule.

Observations:

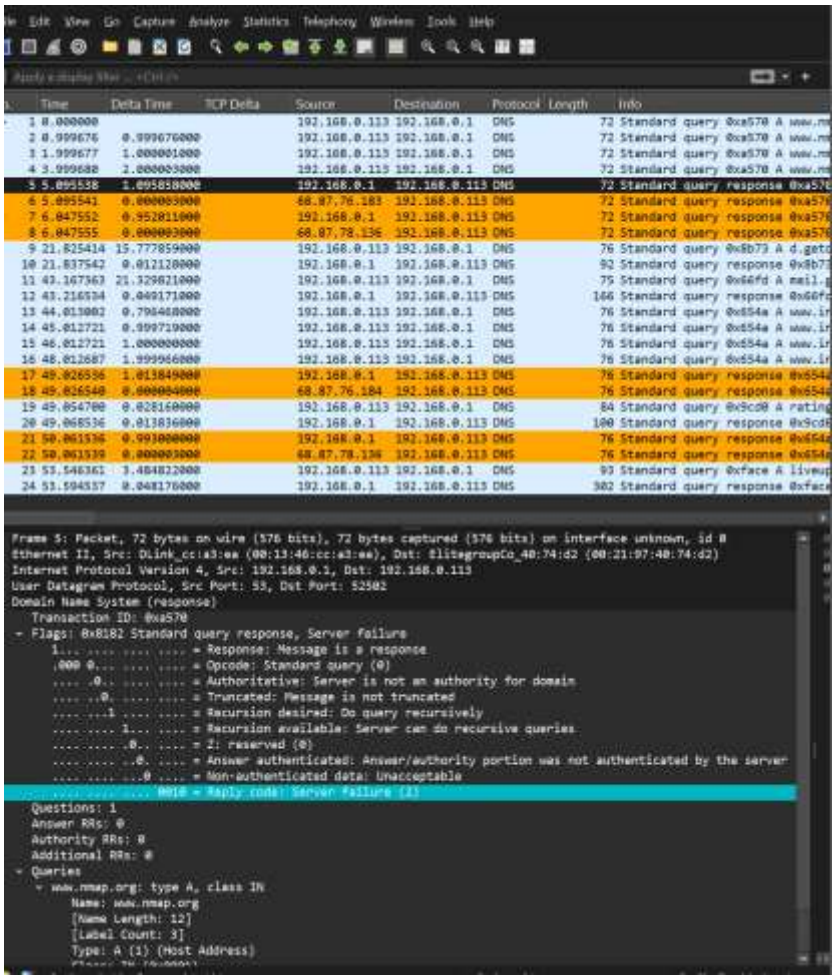

Figure 5.1

The coloring rule filtering DNS flag codes worked as expected (Figure 5.1).

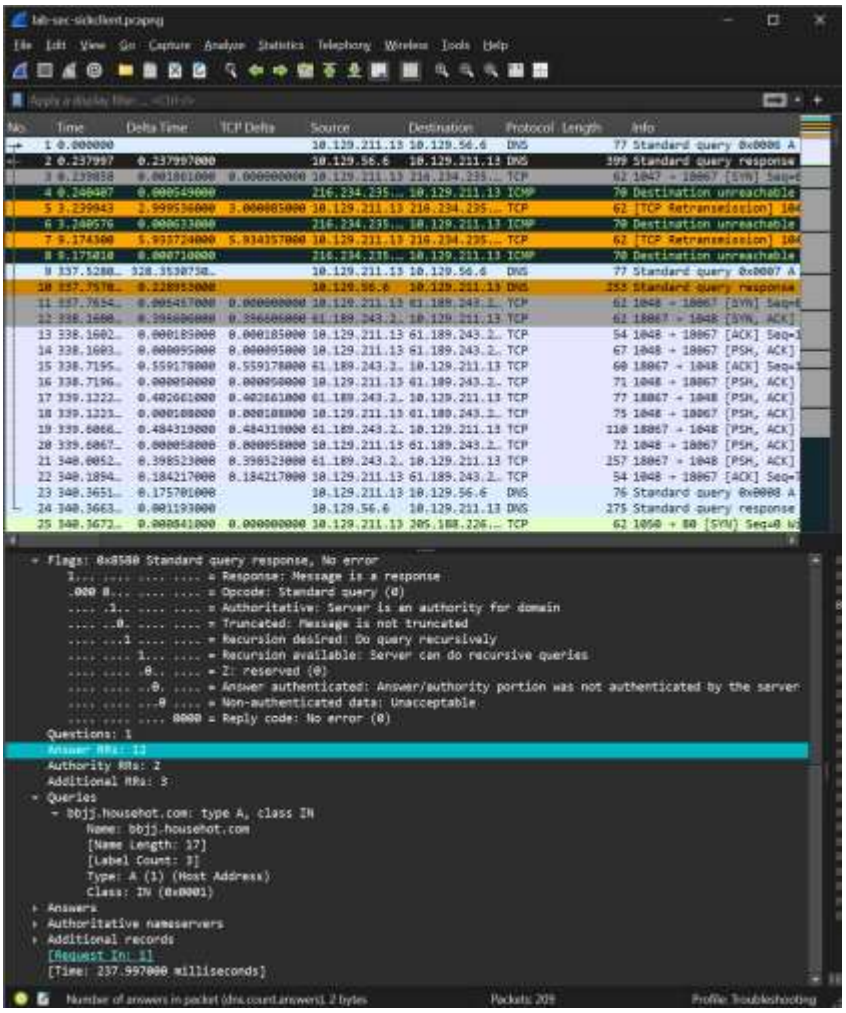

Figure 5.2

After creating a filter to detect high DNS answer counts and opening a new .pcapng file, several packets meet the new coloring filter as expected (Figure 5.2).

Lab 6

Objective:

Practice isolating specific network traffic by applying inclusion and exclusion display filters based on IPv4 address ranges and CIDR notation.

Instructions:

1. Open lab-throughput.pcapng.
2. Identify IP addresses associated with newsrss.bbc.co.uk.
3. Apply a display filter using a CIDR range (e.g., ip.addr == x.x.x.x/24) to see all traffic within that subnet.

4. Apply an exclusion filter (e.g., !(ip.addr == x.x.x.x/24)) to hide that specific traffic.
5. Combine filters to show the subnet traffic plus related DNS queries by adding || dns.qry.name to the inclusion filter.

Observations:

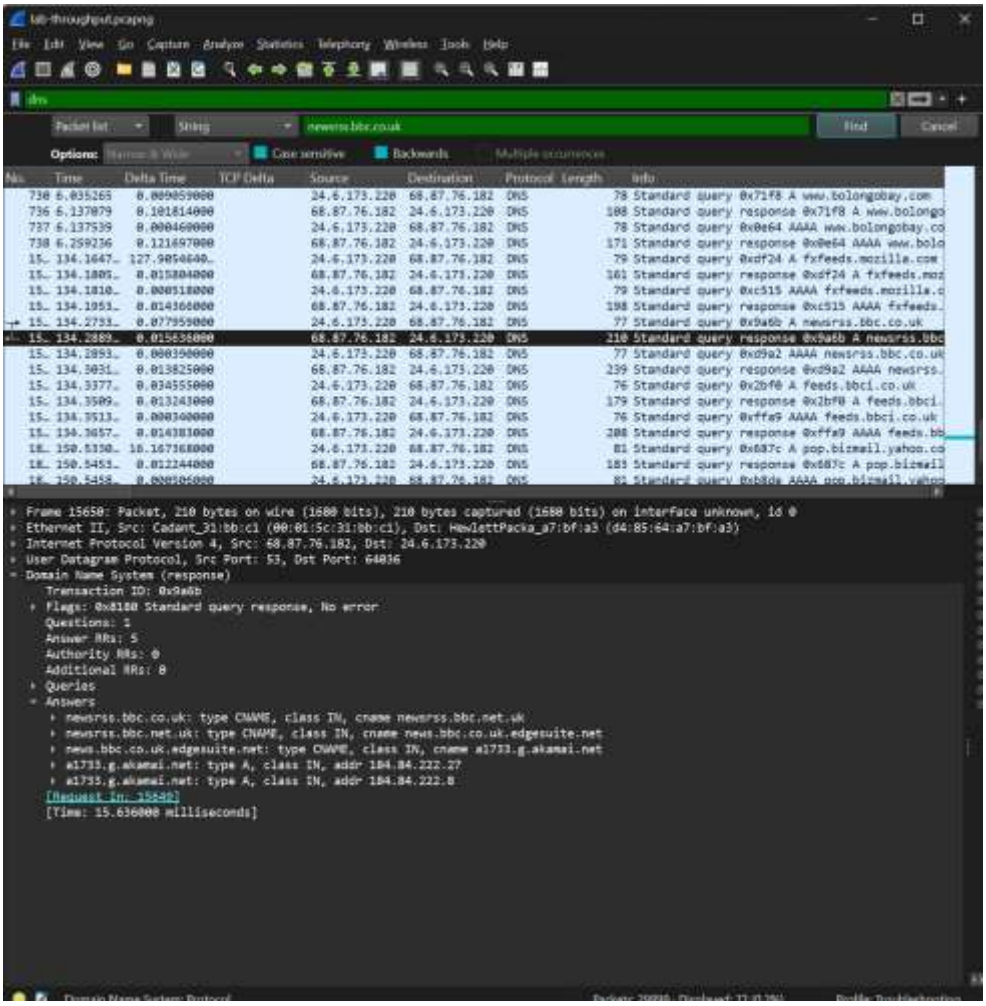

Figure 6.1

The IP addresses associated with newsrss.bbc.co.uk are 184.84.222.27 and 184.84.222.8 (Figure 6.1).

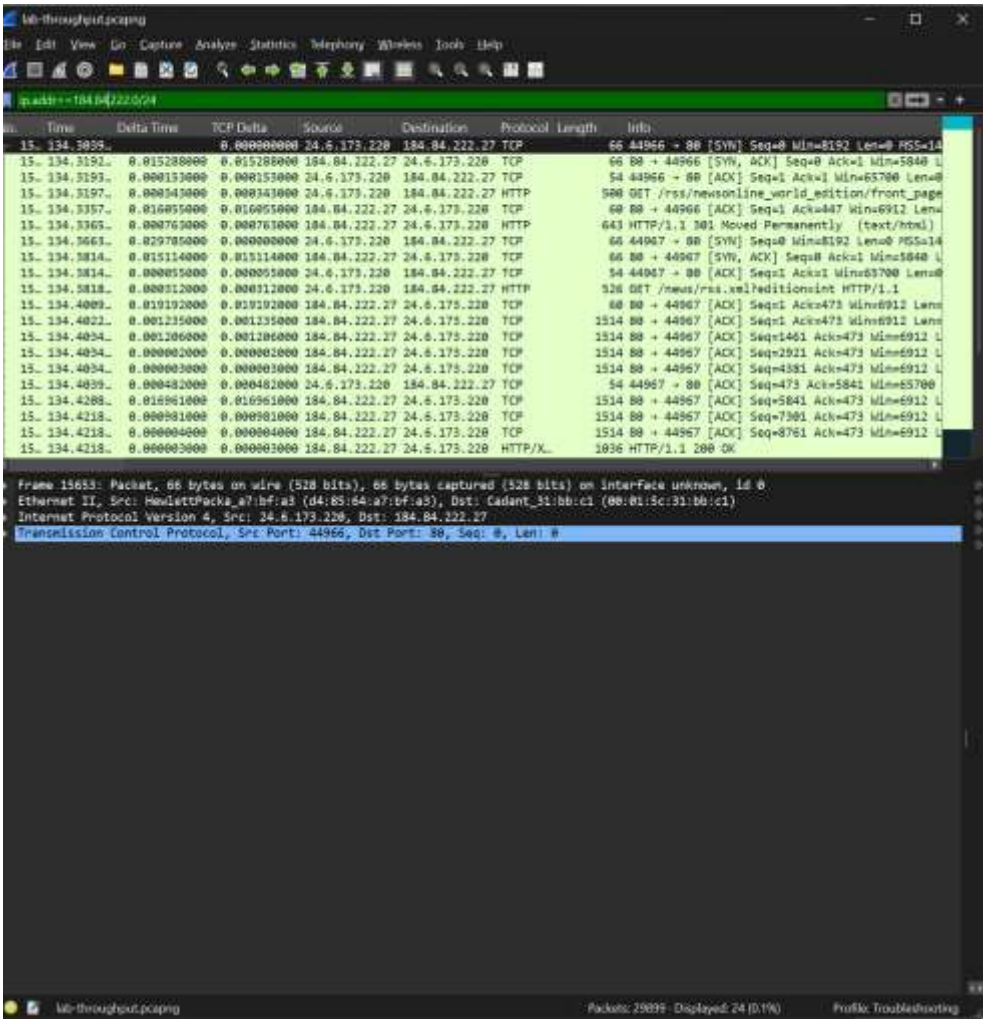

Figure 6.2

After applying the an inclusive IP address filter to see packets associated with 184.84.222.0/24, 24 of the total 29,899 packets matched the inclusive filter (Figure 6.2).

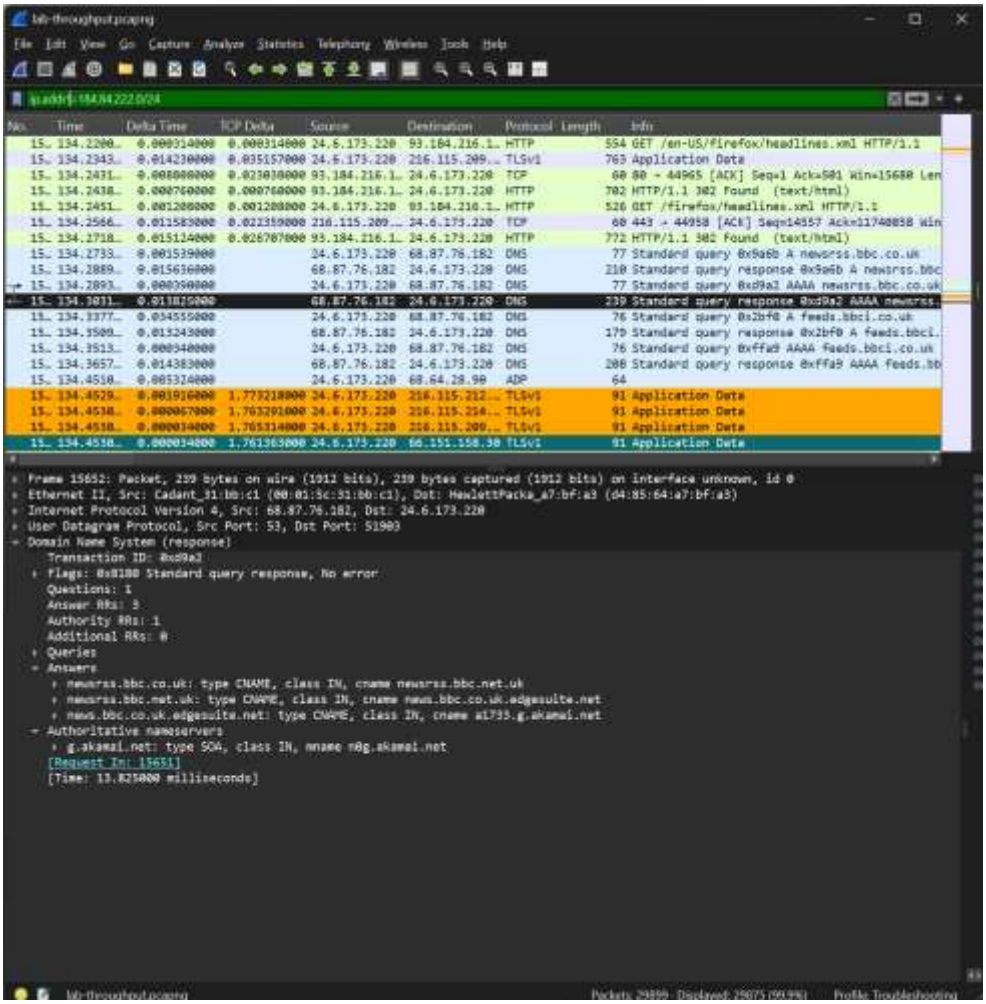

Figure 6.3

Going the other way and applying an exclusive filter using the same IP address range from before 184.84.222.0/24, 29,875 of the total 29,899 packets matched the exclusive filter (Figure 6.3).

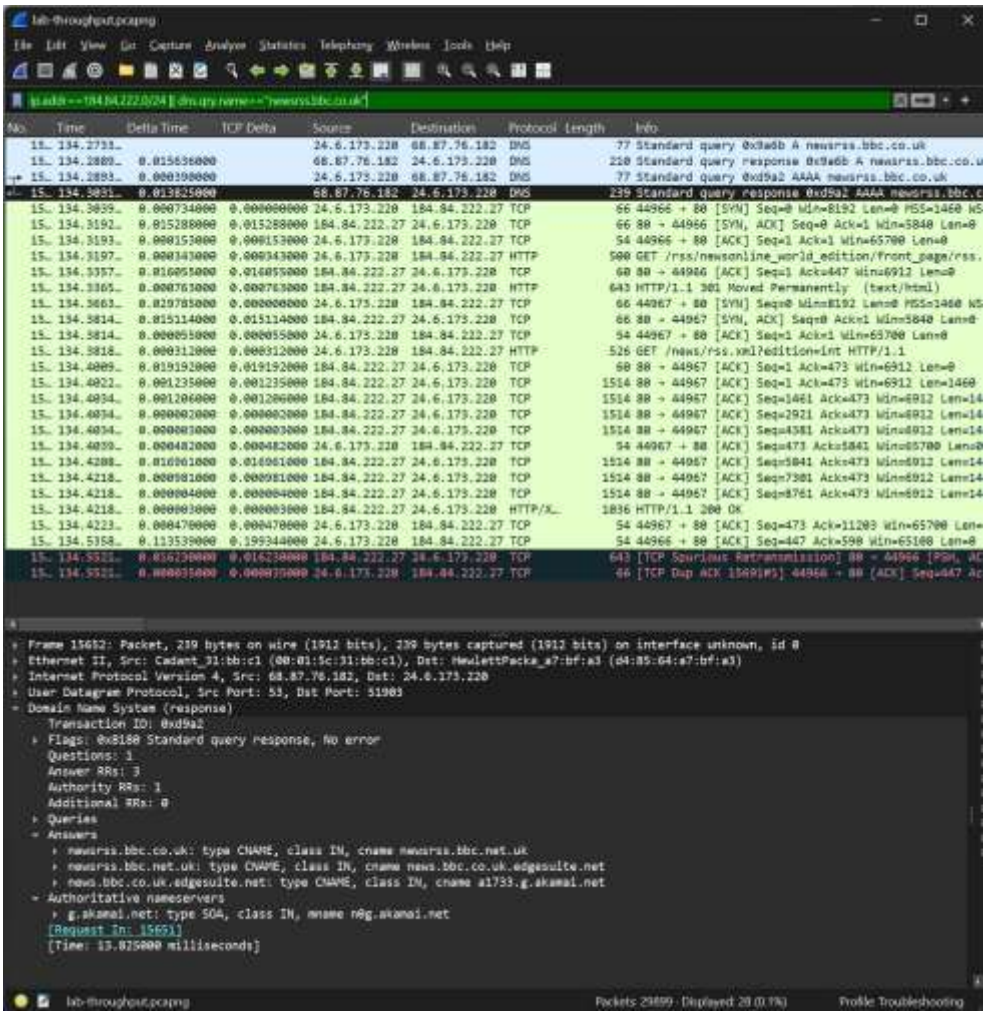

Figure 6.4

The display filter used to display all traffic to and from 184.84.222.0/24 or DNS queries/responses relating to newsrss.bbc.co.uk is ip.addr==184.222.0/24 || dns.qry.name==‘newsrss.bbc.co.uk’ (Figure 6.4).

Conclusion

The methodologies practiced in these labs confirm that a customized, metadata-driven approach significantly reduces the time required to diagnose complex network issues. By isolating specific TCP streams and applying advanced Boolean filters, an analyst can effectively strip away background noise to focus on relevant traffic segments. The integration of visual cues, such as the "T-TCP Delays" and "S-DNS Suspicious" coloring rules, provides an immediate diagnostic layer for identifying performance lags and potential compromises. Ultimately, these skills establish a repeatable workflow for maintaining network integrity and optimizing communication efficiency in diverse enterprise environments.